CandySpeak Manual

Tony Rogvall

2026-07-27

Contents

Introduction

CandySpeak is a reactive programming language designed for embedded systems and microcontrollers. It combines simplicity with powerful abstractions for handling sensors, timers, and I/O.

The language is rule-based - each line describes WHAT should happen and WHEN, not HOW. The system evaluates all rules every cycle and updates outputs based on inputs.

Core Concepts

- **Variables** - store values between cycles
- **Digital I/O** - buttons, LEDs, relays
- **Analog I/O** - sensors, PWM
- **Timers** - time-based events
- **Modules** - reusable components
- **States** - organise rules into state machines
- **Buffers & frames** - shared storage, bit-fields, pack/unpack
- **CAN** - a frame is a buffer with an address; fields are bit views into it

Language Reference

Numbers

```
42              decimal
0x2A            hex
1.5             float
1.2.3.4         an IPv4 address
```

A dotted quad is a number, not a string. `1.2.3.4` is another spelling of `0x01020304` and is interchangeable with it — it exists so an address reads like an address where one is meant:

```
#define GROUND 192.168.1.2
#buffer Tlm:16 out udp 5000 GROUND
```

How many parts there are decides what it is: one is an integer, two are a float, four are an address. Nothing else is — `1.2.3` and `1.2.3.4.5` are refused by name rather than misread, and so is a part above 255.

`..` is untouched: the dot only continues a number when a **digit** follows it, so `Buf[0..15]` is still a range.

An address takes the same path a hex literal takes, so `192.168.1.2` is the bit pattern `0xC0A80102` and is not refused for being past `INT32_MAX` — the same value `0xC0A80102` has always had. Printed back out it is a signed integer like any other, except where the runtime knows it is an address: `/list` shows the `udp` operand as a dotted quad, so the line goes back in as it came out.

Declarations

Declarations start with # and define program resources.

Variables

```
#variable <name>[:<bits>] [<type>] [= <value>]
```

Examples:

```
#variable Counter:8 integer = 0
#variable Temperature float = 20.0
#variable Flag = 0
```

The width is in **bits, 1..32** (32 when omitted); anything else is refused, as is a literal that does not fit the type — a wrong width is far more expensive to find later than at the declaration.

> **A width without a type is SIGNED — except one bit.** `#variable c:4` is a 4-bit *signed* value, so it counts `-8..7` and `c = 15` reads back as `-1`. That follows the rule that a typeless leaf is `integer`, and it applies to `#local`, `#param`, `#constant`, `#field` and `bind` alike. When you want the plain `0..2^N-1` range — flags, bit quantities, raw signals — say so:
>
> ```
> #variable c:4 unsigned = 15      // 15, not -1
> ```
>
> **`:1` is unsigned.** A one-bit signed integer holds exactly `0` and `-1`, and nobody means that: `#variable Flag:1 = 1` would store `-1`, so `Flag == 1` would be false and the flag silently dead. One bit is therefore `unsigned` unless the declaration says `integer`, and that holds for every kind of leaf. Two bits and up keep the signed default, because `{-2,-1,0,1}` is a real range.
>
> ```
> #variable Flag:1 = 1             // 1        (unsigned by default)
> ```

```
#variable Odd:1 integer = 1      // -1       (you asked for it)
#variable c:2 = 3                // -1       (signed, as before)
```

Constants

```
#constant <name> = <value>
```

A constant is a named read-only value fixed at declaration. Unlike a variable it cannot be assigned by a rule; use it for thresholds, scale factors and pin-count limits that should read as names instead of magic numbers.

```
#constant Setpoint = 200
#constant Scale    = 4
```

Parameters

```
#param <name>[:<bits>] [<type>] = <value>
#param <name>[N] [:<bits>] [<type>] = { <value>, ... }
```

A parameter is a **tunable constant**. It reads exactly like a `#constant` — a rule may not assign to it — but it is not folded, because something outside the program is allowed to change it while the program runs. That is the whole difference, and it is visible in a listing:

```
#param Kp:16 = 5
#variable Out = 0
Out = Kp * 2 ? 1
```

```
#param Kp:16 integer = 5  // R
#variable Out:32 integer = 0  // R
Out=Kp*2 ? 1  // 1 R
```

Written with `#constant` instead, the same rule lists as `Out=5*2 ? 1` — the value has been baked into the rule and nothing can move it afterwards.

Use `#param` for the numbers you expect to tune on the bench: gains, periods, thresholds, directions, a board's calibration offsets. Use `#constant` for the ones that are part of the program's meaning, and let them fold.

Setting one A rule may not:

```
Kp = 9 ? 1
Error: cannot assign to a #param in a rule -- set it with > name = value
```

An immediate at the prompt may — that is what makes it a parameter:

```
> Kp = 7
7
```

`/state` lists parameters with the variables, because a parameter's live value is precisely the thing that can differ from what the source says. A plain constant is not listed there; its value is in the listing and nowhere else.

```
Kp                  param           = 7
Out                                 = 10
```

Re-declaring one A parameter — and only a parameter — may be **re-declared**, and that is the mechanism for loading settings from a file: a config file is a list of `#param` lines pasted over a running program.

```
#param Kp:16 = 9
```

The width and type must match the declaration being set:

```
#param Kq:16 = 3
#param Kq:32 = 8
Error: #param Kq does not match the declaration it sets -- same width and type
```

which is not pedantry — any rule already compiled against `Kq` was built for that width. A name held by something that is not a parameter is still an ordinary redefinition error, and arrays are not re-declarable.

This works even when the parameter shipped inside the firmware image, where its value sits in flash and cannot be written. The re-declaration is stored as a RAM **shadow**, and start-up applies it onto the ROM parameter's slot by matching the two **names**.

An overridden parameter lists **once**, as the override, tagged `P` where an ordinary line carries F/E/R:

```
#variable Out:32 integer = 0  // F
#param Kp:16 integer = 9  // P
Out=Kp*2 ? 1  // 1 F
```

The ROM row is hidden because it no longer says what the program runs with, only what it shipped with. `/state` hides the opposite half — it shows the parameter being set, since that is where the live value lives, and not the override's own unused slot.

A shadow is an ordinary RAM declaration, so `/save` persists it with everything else and it is back after a restart:

```
> Kp
9
```

The **shadow** does not survive a changed program. It is a RAM declaration, so it rides in the EEPROM patch, and the patch records a fingerprint of the firmware it was made against — the image's counts and every section CRC. Start-up drops the whole patch when that fingerprint has moved, silently, because a patch belonging to a different program is expected after a reflash rather than an error.

The **setting** does survive it. Setting a parameter also records an entry in a separate store that is keyed by name and not fingerprinted, so the tuning is still there after you add a rule and reflash. That is the store described under Settings, and it is the mechanism to rely on for a value that belongs to the unit rather than to the build.

Where a constant is expected A parameter may stand where a declaration wants a constant — a `#timer`'s period, a `#variable`'s initialiser:

```
#param Period = 1000
#param SD = 7
#timer Tick Period = 1
#variable Pt = SD
```

It cannot *fold* there, for the reason above, so the declaration compiles to two things: the parameter's value as it stands now, plus the assignment that reads the live one at start-up.

```
#param Period:32 integer = 1000  // R
#param SD:32 integer = 7  // R
#timer Tick 1000 = 1  // R
#variable Pt:32 integer = 7  // R
#in INIT  // R
  Tick.period=Period  // 1 R
  Pt=SD  // 2 R
#end   // R
```

That is exactly what such a program had to write out by hand before. INIT is a one-cycle state, so the assignment happens once and the variable is an ordinary variable from then on; inside a module the gate is the *module's* INIT, so each instance is set separately. Consecutive declarations share one block.

The declared value is not zero but the parameter's value at that point, so the one cycle before INIT commits has something real to work with — a timer whose period is still 0 when the runtime first looks fires immediately and stops itself. Only a plain name does this; anything computed leaves the declaration at 0 and waits for INIT.

A `#constant` may **not** be initialised from a parameter. A constant that could change is a contradiction, and the whole point of folding is that the value is final:

```
#constant K = P
Error: syntax error
```

Defines

```
#define <name> <value>
```

A **compile-time** name. The value folds into the code exactly as a `#constant`'s does; the difference is that the name is forgotten once the program is built. It makes no declaration, it never enters the string table, and it does not appear in a generated ROM image.

```
#define ADC_UPPER  0x01
#define ADC_LOWER  0x02
#define ADC_LIMITS  ADC_UPPER | ADC_LOWER    // built from the two above
```

The value is a constant *expression*, so a define may be built from defines declared before it.

Why it exists. Every identifier a program declares is stored, and a declaration's name field is 9 bits — so a program cannot have more than **512 names**, ROM and RAM together. Their total length is a separate limit, set by the string buffer (`STRING_BITS`, 4 KB on the host and smaller on a small part), and a name costs its length plus one: strings are stored as a length byte followed by the characters, with no terminator after them.

The two used to be the same limit, because a name field held a byte offset rather than a handle — 512 *bytes* of names, which a module with a dozen long `#param` names and a set of flag constants reached while still unfinished.

`lib/analog.csp` is the case that prompted this. Its nine `ADC_` flag names cost 167 bytes and contributed **nothing** to the generated code — the compiler had already folded them, so the instructions were identical with or without the names. Moving them to `#define` took the module from 492 bytes of names to 325, which is the difference between having room for the rest of the logic and not. (Those two figures predate the format losing its nul terminator; the module measures 215 bytes today, and the *saving* is what the example is about.)

What you give up. A `#define` does not appear in `/list`, so a listing of a program that uses one cannot be pasted back and give the same program. Keep the source. That is the trade: `#constant` is a declaration you can see and re-read, `#define` is a name that exists only while the compiler runs.

Use `#constant` for a value you want to inspect at the prompt, and `#define` for the flag bits and masks that are only ever a way of writing a number.

Locals

```
#local <name>[:<bits>] [<type>] = <expression>
```

A local **binds a formula**. It is not a variable you assign to; it is a name for a calculation, evaluated once per cycle before the rules below it.

```
#local Gx = 512 - AccX
#local Tilt = (Gx*Gx + Gy*Gy) / 100
```

What makes it worth having is **when** the value becomes visible. Every other leaf follows the transaction rule: a rule reads the value committed at the end of the previous cycle. A local does not — it is readable in the *same* cycle it is written. So a chain resolves in one cycle:

```
#local Sum   = a + b
#local Twice = Sum * 2
#local Plus1 = Twice + 1        // all three settle in ONE cycle
```

Written as three `#variable` rules the same chain would take three cycles, one per step, because each read would see the previous commit.

The inputs are ordinary reads and still follow the ordinary rule — it is the local that is immediate, not what it looks at.

A local must be declared before it is used. There are no forward references, which is also what makes declaration order the evaluation order: no separate phase is needed, and a local can safely refer to locals above it.

`:bits` truncates at the binding point, so a local is a good place to narrow a value once instead of at every use:

```
#local Low:8 = 300              // 44
```

Assigning to a local is an error, reported as such rather than as an unknown name:

```
Gx = 5
Error: cannot assign to a #local -- it binds a formula
```

Scope. A local belongs to the module that declares it. `obj.name` on a local from outside is an error:

```
value1 is a #local -- only visible inside its own module
```

That is what a local *is* — a step in the module's own calculation, not a value that lives somewhere and can be read. Expose it as an `out` variable if the outside needs it.

A local is saved. `/save` keeps it, because a local is a declaration plus the rule that computes it — drop the declaration and that rule has no target. What it does not get is a `/state` row.

A local has no `/state` row. `/state` shows the machine's state; a local is a formula recomputed from that state every cycle, so listing it would be listing an intermediate result. Read it through the variable you assign it to.

A local lists as `$N`, not by name.

```
#local $1:32 integer
$7=$3&-$4|~$3&$6
```

The number is its position among the enclosing scope's locals, generated when the listing is written and stored nowhere — so a local costs no space in the name table, which has a hard 512-name ceiling shared by ROM and RAM (see `#define`). Since nothing outside may name a local anyway, there is nothing for a name in the listing to be used *for*; `$3` says what it is instead of suggesting a handle that does not exist.

> **Listing caveat.** `/list` shows a local's declaration and its formula as two lines — the declaration, and the rule the formula compiled to. That listing does not paste back: the pasted declaration would bind the local to its initial value and the formula line would then be refused. Keep the source.

Digital I/O

```
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
```

Port is optional (for microcontrollers with multiple I/O ports).

Examples:

```
#digital Button in pullup 2
#digital Led out 13
```

```
#digital Relay out B:7
#digital Status inout C:4
```

Analog I/O

`#analog <name>[:<resolution>] [in|out] [pwm] [integer|unsigned] [<port>:]<pin>`

Resolution is number of bits (default 10).

Examples:

```
#analog Sensor in A0
#analog Dimmer:8 out pwm 9
#analog HighRes:12 in A:1
```

> **A reading is SIGNED unless you say otherwise** — the same default a `#variable` has. A sensor that swings both ways about a rest point reads naturally that way: for an accelerometer 0 is level, and there is no midpoint to subtract in every rule.
>
> Say `unsigned` when the value is a bit pattern or a magnitude rather than a quantity — a packed RGB565 pixel sets bit 15 at full red, and reading that back as a negative number helps nobody:
>
> ```
> #analog Pixel:16 out unsigned 9:0
> ```
>
> The value slot is 16 bits wide whichever you choose; the declaration decides how those bits are read back — and how it is CALCULATED with. Seven operators read the sign: `/`, `%`, `>>` and the four order comparisons `< <= > >=`. One unsigned operand makes the whole operation unsigned, so `Index % 10` counts the way you meant it even though `10` is an ordinary literal, and the type survives a chain like `(Index + 1) % 10`. Everything else — `+ - * & | ^ == !=` — gives the same answer either way.
>
> A comparison's RESULT is a truth value and stays signed. And signedness comes from the declaration, not from the literal: a bare `0xFFFFFFF7` is a signed integer, exactly as it would be in C. Give it a type if you need otherwise:
>
> ```
> #constant FULL:32 unsigned = 0xFFFFFFF7
> ```
>
> How a board maps its hardware onto the declared resolution is the board layer's business — see the notes for your target.

Arrays

Any of `#variable`, `#constant`, `#digital` and `#analog` can declare an array by putting a length after the name:

```
#variable Acc[3] = 0
#constant CT[10] = { -100, -81, -31, 31, 81, 100, 81, 31, -31, -81 }
#digital  D[5]  in 0:1..3,7,9
#analog   P[10]:16 out unsigned 9:0..9
```

An array is **N declarations, one per element** — the head keeps the name, the rest are unnamed and follow it. Everything else about the declaration works unchanged: width, type, direction, an initial value.

Constants take an init list. One value per element, in braces:

```
#constant ST[10] = { 0, 59, 95, 95, 59, 0, -59, -95, -95, -59 }
```

Written this way the table is checkable at a glance, which ten separate rules never were.

An element may be any constant **expression**, including one that names a constant declared earlier, and a `string` array takes string elements:

```
#constant Half[3] = { MAX/2, MAX/4, MAX/8 }
#constant Names[3] string = { "off", "on", "auto" }
```

Devices take a pin range, a pin list, or both. Each element carries its own port and pin, so one line describes ten outputs:

```
#analog P[10]:16 out unsigned 9:0..9      // pins 0..9 on port 9
#digital D[5] in 0:1..3,7,9               // pins 1,2,3,7,9 on port 0
#analog Q[3]:16 out 0:1,4,7               // a plain list, no range
```

The pins do not have to sit on one port. A `<port>:` names the port for the pins after it, and it may stand on its own before a comma:

```
#digital E[4] in 0:2,1:5,2:6,3:7          // one pin on each of four ports
#analog  R[9]:16 out 1:1..3,2:1,3,5,9:,7..9
```

The last line reads: pins 1..3 on port 1, then pins 1, 3 and 5 on port 2, then pins 7..9 on port 9 — nine elements over three ports. A listing collapses runs back into ranges and prints a port only where it changes, so what comes back out is the same spec in canonical form.

A length that disagrees with the number of pins is an error, not silently padded — the extra elements would otherwise all point at pin 0, which is a real pin.

Subscripts An element is `A[<expression>]`, and it works on both sides of an assignment:

```
Acc = (CT[Idx]*Gx + ST[Idx]*Gy) / 100
P[(Idx + 1) % 10] = Colour
```

`A` with no subscript is element 0 — a scalar and a one-element array are the same thing.

A constant subscript costs nothing. `CT[3]` resolves to that element's own declaration when the program is compiled, so it is exactly as fast as a plain name, and an index past the end is caught there and then.

A runtime subscript is checked every cycle against the declared length. An index outside the array reports `index out of range` and the access is skipped — the value is wrong, but nothing outside the array is read or written.

Limits

- An array inside a *named object* (`obj.A[i]`) is not supported.
- The reactive graph tracks dependencies per declaration, so a rule that reads `A[i]` is not woken by a write to `A`. Timer- or state-driven programs are unaffected; a `<-` rule over an array is not yet reliable.

Timers

```
#timer <name> <period_ms> [= 1]
```

`= 1` starts the timer automatically at program start.

Examples:

```
#timer Blink 500 = 1
#timer Debounce 50
#timer Timeout 5000
```

Interrupts

A pin declaration can name a **trigger**, and then `.fired` is true for exactly one cycle after it — the same shape `timeout(T)` has.

```
#digital Drdy in falling 2:13
#digital Btn  in pullup rising 2:7

Sample = Imu ? Drdy.fired
```

A trigger is an **option**, in the same place as `in`, `pullup` and `pwm`: an interrupt is a property of how the pin is configured, not a thing of its own. The pin stays the pin, so `Drdy` still reads its level — a program almost always wants both, and on a falling edge the two agree.

Trigger	Means
`rising`	0 → 1
`falling`	1 → 0
`both`	either direction
`high`	asserted for as long as the pin is high
`low`	asserted for as long as the pin is low
`ready`	the device's own event — a conversion finished

Not every part offers all six. An STM32's EXTI has no level trigger at all, so `high` and `low` are refused there rather than turned into an edge. A refused source is never armed and never fires; `/state` marks it with `!` after the trigger so that is visible rather than silent.

Rules never run in interrupt context. The handler sets one bit and returns; the rule runs in the next cycle, in ordinary rule context, exactly as a timer's does. So a trigger does not make a response faster than one cycle — what it buys is that an edge

shorter than a cycle is not *missed*. `rising(X)` is the same edge sampled in software, and misses those.

Trigger words are ordinary names: `#variable ready = 0` is still legal.

See `doc/EVENTS.md` for the board side, and `utils/gen_chips.erl --irq-of <board>` for which pins on a given part can be sources.

Buffers

```
#buffer <name>:<size> [<type>] [in|out] [<transport>]
```

A buffer is a block of storage that variables can map into. A regular variable owns its own value; a buffer is *shared* — several variables can bind to different bit-fields of the same buffer, and the buffer can also be read and written directly by byte. Buffers are the building block for frames (CAN), bit packing, and overlapping data.

```
#buffer Frame:8            // 8 bytes of storage (64 bits)
#buffer Word:2             // 2 bytes
```

> **The size is always in bytes.** A `#buffer` is a byte container, whatever its transport — plain or CAN alike (a frame's size *is* its DLC, also bytes). Need a sub-byte masked value? That is a `#variable` (it carries its own bit width). Need a bit-view into a buffer? That is a `#field`, or a `bind` range.

A buffer may be declared inside a module, in which case every instance gets its own storage — see *What a Module May Contain*.

A buffer can be used directly like a variable. The whole buffer is its value (up to 4 bytes — larger buffers are accessed by byte or through bound fields):

```
#buffer Status:1
Status = 200
```

Byte access. Indexing a buffer selects whole bytes:

```
Frame[0] = 52          // first byte
Frame[1] = 18          // second byte
X = Frame[0..1]        // bytes 0..1 as one value (little-endian)
```

Binding variables (bit-fields). Map a variable onto a *bit* range of a buffer with `bind`. Writing or reading a bound variable reads/writes the underlying buffer bits — they alias the same storage. The buffer must be declared before it is bound.

```
#buffer Frame:3                          // 3 bytes = 24 bits of storage
#variable Speed:8     bind Frame[0..7]   // bits 0..7
#variable Rpm:10 big  bind Frame[8..17]  // bits 8..17, big-endian

Speed = 50             // writes into Frame's bits 0..7
```

> **Note:** when indexing a *buffer* directly the index is a **byte** (`Frame[1]`), while a `bind` range is in **bits** (`Frame[8..17]`). Direct indexing is for convenient whole-byte access; `bind` is for packing sub-byte bit-fields.

`big` on a `bind` means what it means on a `#field`: it selects **MSB-first bit numbering** as well as the byte order, so the same index names a different physical bit. See *Which bit is bit n* under CAN frames — the rule is the same wherever a bit range appears.

Either way the range must stay **inside the buffer** and cover at most **32 bits** — a bound field or a `Buf[a..b]` slice that reaches past the end, or asks for more than 32 bits, is refused at declaration time.

Transports

A buffer with a **transport** is bound to something outside the program. The bytes are the same bytes; what changes is that they arrive from somewhere, or leave for somewhere, on their own.

```
#buffer F201:8  in  can 0x201           a CAN frame
#buffer Imu:14  in  i2c 3 0x68 0x3B     I2C3, device 0x68, from register 0x3B
#buffer Gyro:6  in  spi 1 2:4 0x28      SPI1, CS on port 2 pin 4, command 0x28
#buffer Rx:16   in  udp 5000            listen on port 5000
#buffer Tlm:16  out udp 5000 GROUND     send to GROUND:5000
```

`.rx`, `.tx` and `.dlc` work on all of them — see *Frame parts* below, which is written for CAN and true for every transport.

Two shapes, and the difference is who starts the transfer.

`can` and `udp` are **asynchronous**: packets arrive by themselves and are collected at the top of each cycle. Nothing in the program asks for one.

`i2c` and `spi` are **synchronous** — the program is the master. A transfer is started at the END of a cycle and collected at the start of the NEXT one, so a 14-byte read at 400 kHz costs the loop nothing and the reading is one cycle old. An `in` buffer transfers every cycle, which is what makes a sensor behave like an analog input; an `out` one transfers when a field changed or a rule set `.tx`. Only one transfer per buffer is ever in flight.

The UDP address is a number, and the port comes first.

```
#buffer Tlm:16 out udp 5000 192.168.1.2
```

`192.168.1.2` is a *number literal* — another spelling of `0xC0A80102`, and interchangeable with it everywhere — see *Numbers*. The port leads because it is the half both directions have: `udp 5000` on its own listens, and the optional address after it says where to send.

On an `in` buffer that address is a sender filter, not a destination.

```
#buffer Rx:16 in udp 5000               anyone may send
#buffer Rx:16 in udp 5000 0.0.0.0       the same thing, written out
#buffer Rx:16 in udp 5000 192.168.1.2   only that peer
```

A datagram from anyone else is read off the socket and thrown away — not left there, where it would sit at the head of the queue and stall the port behind it. The filter is exact for now; a real netmask is a later thing.

The filter belongs to the **port**, not to the buffer: with two views of one port (below) the address on the first one is the one that applies.

A broadcast address in that position means a BUS instead.

```
#buffer Rx:8 in udp 5000 192.168.1.255       every node on the subnet
#buffer Rx:8 in udp 5000 127.255.255.255     several nodes on one laptop
```

A broadcast address can never be a *sender* — no datagram arrives from one — so the operand carries both meanings with nothing to tell them apart. On a bus the port is opened so that **several nodes may bind it and each gets a copy** of every datagram, and the filter is off: everyone on the bus is welcome, and who a message is *for* is the program's business.

That reuse is granted **only** for a bus. On unicast it would mean the opposite — the kernel hands each datagram to exactly one of the processes holding the port, so a forgotten node swallows half the traffic and neither end says a word.

> An address ending in `.255` counts as broadcast, as does `255.255.255.255`. A host address never ends in `.255` on a /24.

A datagram that has been overtaken is dropped. An `in udp` buffer holds one datagram, and each cycle the port is drained into it: what the program sees is the NEWEST thing the peer said, and whatever was queued behind it is discarded unread. That is deliberate. A datagram is a snapshot of the sender at the moment it left, so a queue of them is not data waiting to be read but data that was already stale when it was not read — and a peer sending faster than the cycle would otherwise build a backlog that never drains, leaving the program permanently behind reality and falling further behind the longer it runs. `Rx.rx` is true for the cycle after any datagram arrived, whether that was one or twenty.

Use `.rx` to *react* to what arrived, not to count arrivals. If every message has to be seen, a datagram is the wrong carrier for it.

Two `in udp` buffers on the same port are two views of it, not two consumers. The port is bound once, and both buffers parse the same datagram with their own fields.

The console wire A node's serial port feeds its interpreter, and the interpreter prints back to it. Those are the two ends of one wire, and a buffer can splice into either:

```
#buffer Cn:8 inout console      the SERIAL PORT
#buffer Rp:8 inout repl         the INTERPRETER
```

	`in`	`out`
`console`	what was **typed**	what is **shown**
`repl`	what it **printed**	fed in **as if typed**

A node being driven from somewhere else declares `repl`: what its interpreter prints becomes bytes a rule can send, and bytes a rule receives are run as commands. **A**

board with no serial port has a `repl` all the same — that is the point of naming the interpreter rather than the UART.

The node with the keyboard declares `console`, and switches between its own prompt and the far end with **`Ctrl-]`**. It says which way it went:

```
> [remote]        keystrokes now go to the buffer
> [local]         and back to the prompt
```

The escape is not a rule and cannot be one. While diverted, every keystroke belongs to the far end, so the local prompt is unreachable — and if it is the relaying rule that is wrong, there is no way back at all short of a reset. So it is one character compare ahead of everything else, the way telnet's `^]`, minicom's `^A` and ssh's `~.` all are. `/state` says `console remote` while the diversion is on.

A stream, not a frame. Each cycle a buffer takes as many bytes as it holds and leaves the rest for the next one. Nothing is dropped for being overtaken the way a datagram is — a byte has no newer version of itself. What *can* be lost is output printed faster than the rules carry it away: the ring is finite, and when it fills the bytes are dropped and **counted**, because a hole in a relayed listing reads as valid output. `/state` prints `console lost N`.

> **Off by default.** The rings cost `CSP_CONSOLE_BYTES` (256 on the host, 0 everywhere else) — a part with 2K of RAM should not carry them for a transport it never names. At 0 the declarations still compile and run and simply never deliver, like any other bus the target does not have.

A missing bus is not an error. A program using a transport the target does not have compiles, links and runs — it simply never delivers, `.rx` stays false, and rules guarded on it do not fire. That is what lets a program be written and tested on the host and then moved to a board.

CAN frames

A frame is a buffer with an address. Declaring one adds a transport and a frame id to an ordinary buffer; everything else about buffers then applies unchanged — `bind`, `<<=`, `>>=`, byte indexing.

```
#buffer F201:8 in  can 0x201      // 8 BYTES (the DLC), received
#buffer Fbig:64 out can 0x300     // CAN FD, transmitted
```

The direction is the bus direction: `in` frames are received and unpacked, `out` frames are assembled and transmitted. Ids above `0x7FF` are sent as extended (29-bit) frames automatically.

Fields Signals inside a frame are bit views into it. There are three ways to reach them, and they are interchangeable — pick whichever reads best:

```
#buffer F201:8 in can 0x201

// 1. #field -- a named field, declared against the frame
#field Speed:16 unsigned F201[0..15]
```

```
#field Temp:8   unsigned F201[16..23]

// 2. bind -- the same thing spelled as a variable
#variable Rpm:16 bind F201[24..39]

// 3. >>= -- no declaration at all, unpack on the fly
F201 >>= a:8 b:8 c:16 ? F201.rx
```

A `#field` field inherits its direction from the frame, so it rarely needs one of its own. The bit range is in **bits**, counted from bit 0 of byte 0.

Which bit is bit *n* `big` does more than pick a byte order for multi-byte values: **it changes which physical bit each index names.** This matters for a one-bit field, where byte order cannot mean anything at all, and it is the single easiest thing to get wrong when transcribing a signal from a CAN database.

Default (little): indices run **LSB first** inside each byte. With `big`: they run **MSB first**.

So for byte 1 with the mask `0x40` — bit 6 of that byte:

```
#field Sig:1 unsigned      F[14]    // little: 8 + 6      = 14
#field Sig:1 unsigned big  F[9]     // big:    8 + (7-6)  = 9
```

Both name the same physical bit. The conversion between them, within a byte:

```
big_index = 8*byte + (7 - bit)
little_index = 8*byte + bit
```

Which one to use is decided by your tools, not by taste. CAN databases and most bus analysers (DBC, J1939, and anything that shows you a `bytepos` and a `bitmask`) number bits MSB-first, so a signal documented as “byte 1, bit 6” transcribes directly as `big F[9]` and needs arithmetic to become `F[14]`. If the numbers in front of you came from such a tool, write `big` and copy them across; the default is for packing you define yourself.

Because all fields of a frame are views into one buffer, writing one field and reading another shows the packing directly:

```
#buffer Out:8 out can 0x200
#field Lo:8  unsigned Out[0..7]
#field Hi:8  unsigned Out[8..15]
#field Both:16 unsigned Out[0..15]

Lo = 1
Hi = 2                  // Both now reads 0x0201
```

Transmitting An `out` frame is sent at the end of any cycle in which one of its fields changed. That is an **event PDO** and needs no extra syntax — assigning a field is the trigger:

```
Speed = v               // frame 0x200 goes out this cycle
```

For a **cyclic PDO** — send on a schedule whether or not anything changed — there is nothing to make the frame dirty, so ask for the send explicitly with the `.tx` part:

```
#timer Beat 100
Beat = 1
Beat = 1      ? timeout(Beat)
Out.tx = 1    ? timeout(Beat)     // send every 100 ms regardless
```

`.dlc` controls how many bytes go out. It starts at the declared frame size and is clamped to it:

```
Out.dlc = 3             // send 3 bytes instead of 8
```

Receiving A received frame is delivered into the buffer and its fields update. `.rx` is true for **exactly one cycle** — the one in which the received bytes have become readable:

```
println("speed=", Speed) ? F201.rx
```

On the receive side `.dlc` reads back how many bytes the sender actually sent, and `.id` gives the frame id. Both work through the frame or through any field of it (`Speed.id` and `F201.id` are the same fact).

To react only when a *signal* changes rather than on every frame, guard on `changed()` instead — a frame that repeats its contents then prints nothing:

```
println("speed changed") ? changed(Speed)
```

> **One-cycle rule.** `.rx`, `changed()` and `<-` all fire in the cycle where the new value is still in the shadow copy, and rules read the *committed* side. Guarding a print on them directly therefore shows the **previous** value. Route the trigger through a variable to line them up — see *Execution Model*:
>
> ```
> #variable fresh = 0
> fresh = F201.rx
> println("speed=", Speed) ? fresh      // now in phase
> ```
>
> A `binded` variable does not have this problem: it *aliases* the frame bits rather than copying them, so it is already correct in the `.rx` cycle. An unpack (`>>=`) writes copies, which land next cycle like any other write.

Frame parts

Part	Direction	Meaning
`.id`	read	the CAN frame id
`.rx`	read	a frame arrived and is readable this cycle (one cycle only)
`.tx`	read/write	write 1 to force a send; reads back what was requested

Part	Direction	Meaning
`.dlc`	read/write	bytes to send / bytes last received
`.dir`	read/write	bus direction (`in` = 1, `out` = 2)

`.dir` is a property of the buffer, so it also works on a plain `#buffer`.

Note that frame parts live on the buffer rather than in a value slot, so unlike `.period` they are **not** shadowed: `F.dlc = 3` followed by a read of `F.dlc` in the same cycle gives 3.

Connecting to a bus On Linux the frames go to SocketCAN, selected with `--can`:

```
sudo ip link add dev vcan0 type vcan && sudo ip link set up vcan0
./csp --can=vcan0 -c 0 examples/can_input.csp
# from another shell:
cansend vcan0 201#2A3412785634120000
candump vcan0
```

Without `--can` the declarations still parse and the program still runs; frames simply go nowhere. On Arduino the backend is enabled per board by defining `CSP_HAS_CAN` in its Makefile (it expects the arduino-CAN API and a transceiver).

See `examples/can_input.csp`, `examples/can_output.csp` and `examples/can_pack.csp` for worked programs.

> **Limits today.** A field may start at any bit of a 64-byte FD frame (`0..511`) and may be **1..32 bits** wide — the value it produces is a 32-bit container, so a wider signal has to be split into two fields. A declaration outside those bounds, or one that reaches past the end of its frame, is refused rather than wrapped. RTR frames are not supported.

Packing and Unpacking Frames

`bind` gives a *named, permanent* field. When you just want to assemble or take apart a frame on the fly — without declaring a variable per field — use the pack (`<<=`) and unpack (`>>=`) operators. They are the compact, “really cool” way to build and decode frames.

Pack — `<buffer> <<= <field> <field> ...` writes several bit-fields into a buffer in one rule. Fields are **blank-separated** and fill **ascending** bit offsets; each is masked to its width. A field is `<expr>[:<bits>]`; the width defaults to the declared width of the variable when omitted.

```
#buffer Frame:8
#variable A:3 = 5
#variable B:2 = 3
#variable C:3 = 6
```

```
Frame <<= A B C            // 5 | (3<<3) | (6<<5) = 221
```

Fields can be expressions and literals with an explicit width:

```
Frame <<= (X+4):3  X:2  6:3   // 5 | (1<<3) | (6<<5) = 205
```

Unpack — `<buffer> >>= <var> <var> ...` is the exact mirror: each variable receives the next bit-field from the buffer, low offset first.

```
#variable RA = 0
#variable RB = 0
#variable RC = 0

Frame >>= RA:3 RB:2 RC:3   // RA=5, RB=3, RC=6
```

Pack then unpack is a clean round-trip: `<<=` encodes, `>>=` decodes, and the widths line up field for field. Compared to `bind`, pack/unpack keep no state — they are a one-shot encode/decode you place in a rule, ideal for transient frame assembly just before sending or right after receiving.

Modules

Modules group related functionality for reuse. A module is a template that can be instantiated multiple times with different configurations.

```
#module <name>
  <declarations>
  <rules>
#end
```

Module Instantiation

```
#<ModuleName> <instance> [<init>]*
```

Where `<init>` can be:

Form	Meaning
`field = value`	Set the field's **value** (static, runs once)
`field.part = value`	Set a field **attribute** once — `D.pin`, `D.port`, `T.period`
`field <- expr`	Reactive connection (updates when expr changes)

Note the first two are different things: `D = 2` writes a value into `D`, while `D.pin = 2` places the pin. Configuring hardware is always the `.part` form.

Init expressions can be mixed freely. Fields marked `in` must be initialized.

Object Initialisation Semantics The two forms behave differently, by design:

- **Static (=, `.part =`) runs *once*.** These initialisers execute in the instance's implicit **INIT** state and then the object transitions to **NORMAL** (see *States*). Writing them every cycle would be wasteful and would keep configuration outputs permanently "dirty", so they are one-shot.
- **Reactive (<-) is a standing connection.** It re-evaluates whenever an input on its right-hand side changes. It is also **seeded once at start-up**: on the first cycle every `<-` fires once so the field gets an initial value — even when the right-hand side is a constant or a global that never changes afterwards. So `x <- G + 1` gives `x` the value `G + 1` immediately and then tracks later changes to `G`.

Fields versus globals. A `<-` initialiser may read a **global** freely:

```
#M m1 X <- G + 1          // G is a global -> fine
```

To express a relationship *between fields of an object* (or between two instances), write a rule rather than an initialiser — either inside the module, or globally using dot notation on the instances:

```
Total = m1.Out + m2.Out    // relate fields across instances with a rule
```

What a Module May Contain A module is a template. Inside `#module ... #end` you can declare the same resources as at top level — `#variable`, `#digital`, `#analog`, `#timer`, `#buffer`, `#field`, `#constant` — plus module-local `#states`, and the rules (including `#in <state>` blocks) that act on them. Each instance gets its own copy of every declared member and its own state, including its own buffer storage:

```
#module Frame
  #buffer B:16                       // one buffer PER INSTANCE
  #variable lo:8 bind B[0..7]
  #variable hi:8 bind B[8..15]
#end

#Frame f1 lo=1 hi=2                  // f1.B is 0x0201
#Frame f2 lo=9 hi=8                  // f2.B is 0x0809, independent
```

Modules can read globals. A module body sees everything declared above it — constants, variables, timers, buffers. A member with the same name **shadows** the global, so adding a global later cannot break a module that happens to use that name.

```
#constant GREEN = 0x07E0
#buffer Live:16                      // shared by every instance

#module Pixel
  #analog P out 9:0
  P = GREEN ? ...                    // global constant
  P = Live  ? ...                   // global buffer, one for all instances
#end
```

Binding a member to a *global* buffer therefore shares it across instances, while binding to a *member* buffer gives each instance its own — which of the two you get follows

from where the buffer is declared.

Example: Full Adder

```
#module Add
#variable A:1 in
#variable B:1 in
#variable Cin:1 in
#variable S:1 out
#variable Cout:1 out

S = A ^ B ^ Cin
Cout = (A & B) | (Cin & (A ^ B))
#end

#Add a0 A=1 B=1 Cin=0
#Add a1 A=0 B=0 Cin <- a0.Cout
#Add a2 A=0 B=1 Cin <- a1.Cout
```

Here `a0` has static inputs, while `a1` and `a2` chain their carry input reactively from the previous adder's carry output.

Pin Assignment for I/O When a module contains `#digital` or `#analog` declarations, the pin can be left at a placeholder in the module and assigned per instance with the `.pin` part:

```
#module Button
#digital Pin in pullup 0     // placeholder pin
#variable Out = 0
#timer T 50

T = 1 ? Pin != Out
Out = Pin ? timeout(T)
#end

#Button btn1 Pin.pin=2       // assign the PIN at instantiation
#Button btn2 Pin.pin=3
#Button btn3 Pin.pin=4
```

> **`Pin = 2` is not the same thing.** A bare `field = value` sets the field's **value**, exactly as it does for a variable — for a 1-bit digital, `Pin = 2` writes the value 1 and leaves the pin at 0. Use `Pin.pin = 2` to place it, and `Pin.port = 1` for the port. The same applies to `#analog`.

This makes modules reusable across different hardware configurations. It also overrides a default given in the module:

```
#module Led
#digital Out out 13          // default pin 13
...
```

```
#end

#Led led1                    // uses default pin 13
#Led led2 Out.pin=12         // override to pin 12
```

Accessing Module Fields Use dot notation to access a module instance's fields:

```
MainLed = btn1.Out       // read debounced output
```

Rules

Rules have the form:

```
<actions> [? <condition>]
```

Actions are executed when the condition is true. The `? <condition>` part is optional — a rule with no condition runs every cycle (it is always true).

Assignment

Regular assignment - evaluated every cycle:

```
Led = 1 ? Button == 0
Counter = Counter + 1 ? Timer
```

Reactive Assignment

With `<-` the rule only runs when a variable on the right-hand side changes:

```
Output <- Input * 2
```

The rule above runs only when `Input` changes, not every cycle.

Start-up seed. A `<-` rule is also evaluated **once on the first cycle**, so the target always gets an initial value — even if the right-hand side is a constant or an input that never changes afterwards. This means `Output <- Input * 2` is correct from the very first cycle, not just after the next change to `Input`.

With condition - the rule runs when RHS variable changes AND condition is true:

```
Filtered <- RawValue ? RawValue > Threshold
```

Multiple Actions

Separate with comma:

```
Led = 1, Counter = Counter + 1 ? Button == 0
```

Conditions

Conditions can be:

- Comparisons: `==`, `!=`, `<`, `>`, `<=`, `>=`

- Logical: && (and), || (or), ! (not)
- Timer functions: `timeout(T)`, `elapsed(T)`, `progress(T)`
- Special functions: `changed(x)`, `rising(x)`, `falling(x)`

Examples

```
Led = ~Led ? timeout(Blink)
```

Toggle LED at each timeout.

```
Output = 1, T = 1 ? Input && !Output
```

Set Output and start timer T when Input goes high.

States

States turn a flat list of rules into a state machine. You enumerate the states, then group rules under the state they belong to. This keeps large programs readable and lets the runtime skip whole blocks of rules that are not active.

```
#states A B C
```

Three states always exist implicitly: **INIT** (entered at start-up, for one-time initialisation), **NORMAL** (the default running state) and **FAILSAFE** (see below). Your own states are added on top; you can list more at any time with another `#states` line.

Rules are attached to a state with an `#in <state> ... #end` block:

```
#in A
    X = 1 ? Y < Z              // stay in A while this holds
    State = B ? X > Z          // transition to B
#end

#in B
    Z = Z + 1, State = A   // do work, then go back to A
#end
```

A transition is just an assignment to the reserved field `State`.

INIT runs exactly once. At the end of a cycle spent in INIT, `State` steps to NORMAL by itself, so an `#in INIT` block is setup and not a loop:

```
#in INIT
    Count = 0              // runs on the first cycle, and only then
#end
```

An explicit assignment wins — the automatic step is the default for an INIT that says nothing about where to go next:

```
#in INIT
    Count = 0, State = Red   // lands in Red, not NORMAL
#end
```

Assigning `State = INIT` runs the block again, once. Every object gets its own `State`, so `obj.State = INIT` re-initialises that instance on its own.

Mental model. To reason about behaviour, read `#in S` as adding `&& State == S` to every rule in the block. The two forms below behave the same:

```
#in A
    Y = 1, State = B ? X > Z
#end
```

```
Y = 1, State = B ? X > Z && State == A
```

But it is a mental model, not a source rewrite. The block compiles to a **gate** in front of the group: one state test that jumps over the whole block when the state does not match, so an inactive state costs a single check no matter how many rules it holds — that is the point of grouping them. A per-rule state test is kept alongside it for the reactive path, which reaches rules individually rather than by walking the block.

You may add as many rules to a state as you like, and split a state across several `#in S` blocks — they accumulate.

Several states at once. List more than one state to run a block in *any* of them — the states OR together. Shared infrastructure (a heartbeat, a panic button, a timer restart) that must run across several phases goes here instead of being copied into each block:

```
#in red redyellow green yellow
    Phase = 1 ? timeout(Phase)     // restart the timer in every driving phase
    State = FAULT ? Panic          // the panic button, checked in all of them
#end
```

Read `#in A B C` as `&& (State == A || State == B || State == C)` on every rule in the block.

Rules with no `#in` — NORMAL+. A top-level rule that is *not* inside any `#in` block runs by default in the two built-in operating states, **INIT** and **NORMAL** — never in a special state. So a program with no states at all just runs (it sits in INIT/NORMAL), and the moment a state machine steps into a *special* state — a user state, or the reserved **FAILSAFE** — the loose global rules **quiesce** instead of leaking into it. That is what keeps FAILSAFE an island: only `#in FAILSAFE` (and blocks that explicitly list it) run there. To run a global rule in a special state too, name that state with `#in`.

FAILSAFE is sticky. It is the designated safe state, and once `State` holds it **no rule can leave it** — only a reset does. A rule that assigns something else is simply ignored, so a flaky guard cannot bounce the device back out of a safe configuration. Together with the quiescing rule above, that is what keeps FAILSAFE an island: loose global rules stop running there, and only `#in FAILSAFE` (plus blocks that name it) has any say over the outputs.

> This is the shape FAILSAFE has today, and it is deliberately thin. The intended form is a `#module FAILSAFE`, compiled as its **own ROM image** with its own declarations, its own `#in INIT` and its own EEPROM bank — so a device can carry several banks, one of them the safe one, and switching

is a rebase rather than a state transition. Write safe-state logic as a self-contained block now and it will move over cleanly.

States in modules. A module can declare its own local states. Each instance carries its own `State`, so instances step through the machine independently:

```
#module Blinker
  #digital Led out
  #timer   T 500
  #states  on off

  #in INIT
    T = 1, State = off
  #end
  #in off
    Led = 1, State = on ? timeout(T)
  #end
  #in on
    Led = 0, State = off ? timeout(T)
  #end
#end

#Blinker b1 Led.pin=12
#Blinker b2 Led.pin=13
```

Blocks

Rules that share a condition can share it once:

```
#when Drdy.fired && Level > 0
  Sample = Imu
  Count  = Count + 1
#end
```

`#when <condition> ... #end` gates the whole block: the rules inside run only in a cycle where the condition holds, exactly as if each carried it as its own `?` clause. The condition is evaluated **once per cycle** rather than once per rule.

`#in <state>...` is the same shape for the state machine, and the two are kept as separate words on purpose — `#in Idle` reads as “in this state”, and one word with two senses makes both harder to read.

Blocks nest, up to four deep, and `#in`, `#when` and `#module` share one stack — so `#end` always closes whichever opened last:

```
#when Drdy.fired
  #in Armed
    Shot = 1
  #end
  Seen = Seen + 1
#end
```

A block left open at the end of a file is an error, reported by the line it opened on. At the prompt it is not: a block is open there while its rules are being typed, and a bare expression inside one is a rule rather than a query.

Parts

Every resource carries not just a *value* but a set of **attributes** — the pin of a digital, the period of a timer, the direction of a port. These attributes are reachable with dot-notation as **parts**, and most can be both read and written by a rule.

`<resource>.<part>`

Part	Applies to	Meaning
`.val`	any	the plain value (the default when no part is given)
`.pin`	digital / analog	pin number
`.port`	digital / analog	port number
`.dir`	digital / analog / buffer	direction (in/out)
`.pwm`	analog	PWM flag
`.endian`	bound field / `#field`	bit numbering and byte order (`0` native, `1` little, `2` big)
`.pullup`	digital	pull-up enable
`.pulldown`	digital	pull-down enable
`.period`	timer	timer period in ms
`.fired`	timer / interrupt pin	fired this cycle (edge-triggered — one cycle)
`.id`	CAN frame / field	the frame id
`.rx`	CAN frame / field	a frame arrived and is readable this cycle
`.tx`	CAN frame / field	write 1 to force a send
`.dlc`	CAN frame / field	bytes to send / bytes last received

A CAN field answers for its frame, so `Speed.id` and `F201.id` are the same fact. Frame parts are stored on the buffer rather than in a value slot, so unlike the others they are **not** shadowed — writing `.dlc` and reading it back in the same cycle gives the new value.

Examples — reading and writing attributes like any other value:

```
D.pin    = 17              // reassign a digital's pin at runtime
T.period = 500             // change a timer's period
Per      = T.period        // read it back
Ready    = 1 ? T.fired     // use a timer part in a condition
```

Writing `.pin`, `.port`, `.dir`, `.pullup` or `.pulldown` reconfigures the hardware — the pin is put into its new mode at the end of the cycle, so the next read or write uses it.

`.pin` does not release the pin it leaves. A digital that was driving pin 4 high and is then moved to pin 7 configures pin 7 and leaves pin 4 an output, still high — and no name in the program refers to it any more, so nothing can put it right. Set the old pin to a safe level before moving, or do not move it. The same applies to `.port` on a board that addresses pins per port.

Setting a part in object-init is the idiomatic way to configure an instance:

```
#Blinker b1 Led.pin = 12   // configure the instance's pin once, in INIT
```

Execution Model

A single cycle works like this.

- **Cycle.** The runtime repeatedly reads inputs, evaluates rules, and writes outputs. One pass is a *cycle*; `cycle()` returns its number.
- **Atomic commit.** Within a cycle, reads see the values committed at the end of the *previous* cycle, and writes are collected in a shadow copy. At the end of the cycle all changes are committed at once. So the order in which rules are written does not change the result, and a rule never sees another rule's half-finished update. When two rules write the same field in one cycle, the last write wins at commit.
- **Change set.** The commit records which fields actually changed. That is what drives `changed(x)` and the `<-` reactive trigger, and what the reactive execution mode uses to decide which rules to re-run.
- **Sequential vs reactive.** Sequential mode evaluates every rule each cycle. Reactive mode evaluates only rules whose triggers fired. The dependency graph is built from what stands behind `?`, and from both sides of `X <- Expr ? Cond`. A rule with neither a condition nor `<-` — a bare `Y = X` — therefore has **no trigger** and runs only in the first (seeding) cycle. That is by design, not a limitation: reactive mode runs what you told it to watch.

The one-cycle rule. Reads see the previous commit; writes land at the next one. Everything that *triggers on change* — `changed(x)`, `<-`, a frame's `.rx` — fires in the cycle where the new value is still in the shadow copy. So in that cycle the value itself still reads as the **old** one:

```
X = 1                       // X changes 0 -> 1
println(X) ? changed(X)     // prints 0, not 1
```

This is the transaction model applied consistently, not a special case: an input sampled this cycle becomes readable the next one, and that holds for a changed variable too. Two consequences worth knowing:

- To act on a change *with the new value*, route the trigger through a variable. It is delayed by exactly as much as the value, which puts them back in phase:

  ```
  #variable fresh = 0
  fresh = changed(X)
  println(X) ? fresh          // prints 1
  ```

- A source that changes **exactly once** never re-triggers, so `Y <- X` captures the pre-change value and keeps it. With a continuously changing source the same mechanism just shows up as a one-cycle lag, which is harmless.

Values that *alias* rather than copy are exempt: a `binded` variable is the same storage as its buffer, so it is correct immediately.

Built-in Functions

Function	Description
`timeout(T)`	True for one cycle when timer T expires
`elapsed(T)`	Milliseconds since T started
`progress(T)`	0-100, percentage of timer period elapsed
`changed(x)`	True if x changed this cycle
`rising(x)`	True on 0→1 transition in digital input
`falling(x)`	True on 1→0 transition in digital input
`cycle()`	Current cycle number
`tick()`	Current time in milliseconds
`abs(x)`	Absolute value (integer or float, follows the argument)
`sign(x)`	Sign of x: -1, 0 or 1
`min(a,b)`	Minimum value (integer or float, follows the arguments)
`max(a,b)`	Maximum value (integer or float, follows the arguments)
`clip(x,lo,hi)`	Clamp x to the range lo..hi (integer or float)
`trunc(x)`	Float to integer, toward zero
`round(x)`	Float to integer, nearest (halves away from zero)
`print(x)`	Print value (up to 4 arguments)
`println(x)`	Print with newline (0 to 4 arguments)
`latch(b)`	Hold outputs (1) or release (0)

Operators

Priority	Operator	Description
Highest	`~ ! -`	Bitwise NOT, logical NOT, negation
	`* / %`	Multiplication, division, modulo
	`+ -`	Addition, subtraction
	`<< >>`	Bit shift
	`< <= > >=`	Comparison
	`== !=`	Equality
	`&`	Bitwise AND
	`^`	Bitwise XOR
	`|`	Bitwise OR
	`&&`	Logical AND
Lowest	`||`	Logical OR

Linux Tool

Installation

```
git clone <repo>
cd candyspeak
make
```

Usage

```
./csp [options] [file.csp]
```

Options

Option	Description
`-h`	Show help
`-i`	Interactive mode
`-n`	Parse only, don’t execute
`-C`	Show compiled C code
`-c N`	Max number of cycles
`-T MS`	Max runtime in milliseconds
`-r`	Reactive mode (off unless given)
`-Q`	Trace variable values
`-P`	Debug parser
`-R`	Debug result
`-S`	Debug scanner/tokenizer
`-d`	Debug (general)
`-L erlang`	Output trace in Erlang format
`-s <file>`	Write a per-cycle state trace (Erlang format) to a file
`-p <file>`	Write parser debug output to a file
`-O <file>`	Write an object dump to a file
`-e <file>`	EEPROM (persisted state) file to use (default `eeprom.db`)
`--no-eeprom`	Do not overlay the saved EEPROM patches at boot
`-I <file>`	Feed inputs from a file (real time, cycle-stamped rows)
`-F <file>`	Feed inputs with **simulated time** (see below)
`-b`	Start paused; inspect, then `/resume` (implies `-i`)
`--can=IFACE`	SocketCAN interface for `#field` frames (e.g. `vcan0`)
`-m N[k]`	Usable code-memory budget in bytes
`-M N[k]`	Total RAM the simulated board has
`-U N[k]`	RAM the system and linked libraries take
`-E N[k]`	Simulated EEPROM capacity (0 = unbounded)
`--board=NAME`	Simulate a measured board: `mega`, `mkrzero`

Simulating a Board

The host tool can pretend to have a microcontroller's memory, so `/memory` reports numbers that mean something before you flash anything:

```
./csp --board=mega -i program.csp     # measured RAM/EEPROM for an ATmega2560
./csp -M 8k -U 2700 -i program.csp    # or set the numbers by hand
```

`--board` fills in `-M`, `-U` and `-E` from figures measured on real firmware builds (regenerate them with `make boards`). `/memory` then shows how much of that board's RAM the program would actually claim.

Overriding sys.Id and sys.Name

```
./csp --id=7 --name=Node7 prog.csp
```

Both go in as **immediates**, exactly as typing `> sys.Id = 7` would. So they are recorded in the settings store *in RAM* — they survive a rebuild and show in `/settings` — and they are **UNSAVED**: nothing is written to the EEPROM file unless you ask for it with `/save`.

That is what makes them useful for running several nodes against one another on one machine: each gets its own identity for the run, and no test rewrites the store.

The EEPROM Is Read at Start-up

Started **without a program file**, `./csp` behaves like a board coming out of reset: it loads the baked-in ROM program and then overlays whatever `/save` wrote to the EEPROM file (`eeprom.db` unless `-e` says otherwise), so the declarations, rules and disables you saved last time are back. Give it a program file and the overlay is skipped — naming a file means "run *this*". `--no-eeprom` skips it too, for a deliberately clean boot or a repeatable test.

Examples

Run a program:

```
./csp program.csp
```

Show compiled code:

```
./csp -C -n program.csp
```

Run max 100 cycles with tracing:

```
./csp -c 100 -Q program.csp
```

Interactive mode:

```
./csp -i
```

Build a binary that carries a program as its **firmware ROM image** — what a board runs, and the only way to exercise anything that needs a program in flash (the `F` tag in a listing, overriding a `#param` that shipped with the image, the section CRC):

```
make rom PROG=examples/cpx_rotate.csp           # -> tmp/cpx_rotate
make rom PROG="lib/pid.csp app.csp" OUT=tmp/demo # several files, one program
make rom PROG=examples/blink.csp TIER=min       # the smallest board tier
```

`./csp` itself deliberately carries no program, so a listing off it is all RAM. For a real board the image has to be `rom.c`: `make rom-image PROG=…`.

Simulated Time (`-F`)

With `-F` the program runs against a **virtual clock** instead of the wall clock, which makes timer and input timing fully deterministic and instant (no real waiting). Each input row is stamped with an absolute virtual time in milliseconds:

```
<time_ms>  <var>=<value>  [<var>=<value> ...]
```

A row is applied once the virtual clock reaches its time. Between rows the clock jumps straight to the next event (the next input row or the next timer timeout), so a 1-second timer fires after one step rather than a thousand cycles — while still advancing at least one tick per cycle so `cycle()` and time-reading logic always see time move forward.

Example — feed a sensor at two virtual times and let a timer expire:

```
# stimulus.dat
30   Sensor=10
70   Sensor=21
```

```
./csp -c 100 -s trace.txt -F stimulus.dat program.csp
```

`csp_time_ms()` returns the virtual clock in this mode, so `timeout(T)`, `elapsed(T)` and `progress(T)` all behave deterministically. This is the recommended way to write repeatable tests for timers and analog/digital input.

A row carries three kinds of item, and they can be mixed:

Form	Sets
`<name> = <value>`	the value
`<name>.<part> = <value>`	a part — `.pin`, `.port`, `.period`, `.dlc`, `.tx`
`can <id> <byte>...`	delivers a CAN frame

A part is written to **both halves** of the transaction, the way a saved setting is: one written only to the output half would read back from an input half still holding the declared value.

```
# stimulus.dat
0     Sensor.pin=7  Blink.period=250
30    Sensor=10
60    can 0x201 0x03 0x2a          # frame 0x201, two bytes
```

> **A frame takes two cycles to become visible.** The row queues it, `csp_can_recv` takes it on the next cycle, and the commit after that raises `.rx` — the same two cycles a real driver costs. Meanwhile the clock jumps

to whichever comes first, the next pending timer or the next row, so two rows a minute apart can be *adjacent cycles*. When a test sends several commands in sequence, put a row between them to give each frame its cycles; otherwise the clock leaps past the delivery and the second command overwrites the first.

`.rx` itself cannot be set from a row, and should not be: a frame having arrived is a fact about the bus, not a value. Writing it directly would also skip the shadow-heap write and `can_mark_fields`, so `changed()` on the frame's fields would be wrong.

Generate Embedded Code

To generate C code for Arduino or other microcontrollers:

```
./csp -C -n program.csp > program_code.h
```

Then include this header in your Arduino project along with `csp.h` and the runtime files.

Advanced Concepts

Execution Modes

By default every rule is evaluated each cycle. Reactive mode is an optional optimisation for large programs where few things change per cycle.

Regardless of mode, changes within a cycle are applied atomically at its end (see *Execution Model*): rules never see each other's half-finished updates and evaluation order does not matter.

Reactive Mode (`-r`)

Default: OFF — pass `-r` to turn it on.

In reactive mode, only rules whose inputs have changed are evaluated. This is more efficient when:

- Few variables change each cycle
- Program has many rules
- System has limited CPU

It costs some memory for the dependency graph.

What gets a trigger. The graph is built from what stands behind `?`, and from both sides of `X <- Expr ? Cond`. A rule that has neither a condition nor `<-` is not watching anything, so in reactive mode it runs only in the first (seeding) cycle:

```
Y = X           // sequential: every cycle. reactive: first cycle only.
Y <- X          // reactive: whenever X changes
Y = X ? cond    // reactive: whenever cond's inputs change
```

Write the rules the reactive way and both modes reach the same committed state. Rule order is honoured in both: when two rules write the same field, the one written last wins — which is what makes patching a running system predictable.

```
./csp -r program.csp     # reactive mode
./csp program.csp        # evaluate all rules (default)
```

Api

The support configuration and default values for reactive mode are contained in csp_config.h

```
#include "csp_config.h"
```

The configuration parameters are listed below. Each must be defined to either 1 or 0.

```
REACTIVE_DEFAULT
SUPPORT_REACTIVE

USE_STATISTICS
USE_FIXPOINT
```

The programmer's API to change these at runtime — for example in an Arduino sketch during `setup()`:

```
extern int     csp_set_reactive(csp_rt_t*, int onoff);
extern int     csp_set_latch(csp_rt_t*, int onoff);
```

Interactive Mode

Start interactive mode with `-i`:

```
./csp -i
./csp -i program.csp    # load program first
```

Interactive Commands

Command	Description
`/help`	Show commands
`/list`	List declarations and rules, each tagged `F`/`E`/`R`
`/state`	Show current values, grouped per object
`/memory`	Show RAM usage per category (how much space is left)
`/reset`	Reset to initial values
`/clear`	Drop RAM patches, keep the ROM program
`/pause`	Freeze execution; the prompt stays live
`/resume`	Continue (rebuilds first if the program was edited)
`/live`	Freeze the *rules* but keep I/O running
`/latch on`	Hold outputs (freeze current values)
`/latch off`	Release outputs (normal operation)
`/commit`	Commit pending values

Command	Description
`/settings`	Show stored settings — see Settings
`/save`	Save state to storage (EEPROM file)
`/load`	Load state from storage (EEPROM file)
`/quit` (or `/exit`)	Exit

`/state` opens with a status line showing where you are:

```
cycle 214   latch on   running
```

The last field is the run mode — `running`, `paused` or `live`.

Each row is `NAME  DIR  KIND  WHERE  = VALUE`, and what `WHERE` holds depends on the kind — a timer has no pin, a buffer has no pin either:

```
Led          out     digital  0:13    = 1
Beat         running timer    500/174
Tx           out     buffer   0x201/8 = 07 00 00 00 00 00 00 00  TX
Rx           in      buffer   0x200/8 = 0C 00 00 00 00 00 00 00  RX
TxSeq        out     field    [0..15] = 7
```

- a **pin** shows `port:pin`
- a **timer** shows `period/remaining`, and `FIRED` on the cycle it fires
- a **buffer** shows `id/dlc` for a CAN frame (nothing for a plain RAM buffer) and its value is the frame itself, byte by byte. `RX` means a frame landed this cycle, `TX` that one goes out at the end of it.
- a **field** shows the bit window it is a view into

How long a line may be

The line buffer is carved from the same pool as the program, sized to a 32nd of it (never below 64 characters, never above 512). So the limit is a property of the board, not of the build: a mega takes 95 characters, a board with room takes 511. `/memory` has a `line` row showing the current size.

A line past the limit is **refused**, not truncated:

```
Error: line too long, max 95 characters -- line ignored
```

Running a shortened line would be worse than refusing it — `#disable 12` cut to `#disable 1` is not a partial command, it is a different one.

Reading a listing

Every `/list` line ends in a comment carrying the rule number and one letter saying **where that line lives**:

```
> /list
#digital Led out 0:13  // F
#timer Beat 500 = 1  // E
#variable Seq integer = 0  // R
```

```
Beat=1 ? timeout(Beat)  // 1 E
Seq=Seq+1 ? timeout(Beat)  // 2 R
```

Tag	Where it lives	What `/clear` costs you
`F`	Firmware flash — the ROM image	nothing; it is not a patch
`E`	RAM, and EEPROM holds a copy	nothing permanent — `/load` or the next boot brings it back
`R`	RAM only	**the line is gone**
`P`	a `#param` re-declared over one that shipped in flash	**the tuning is gone** unless it was saved; the shipped value is back
`S`	a **setting** overrides this declaration — see Settings	nothing; settings are a separate store and `/clear` does not touch them

`E` and `R` are both RAM patches and look identical everywhere else; the letter is the only place the difference shows. It is the answer to "what do I lose if I type `/clear`" — and to "did my `/save` actually take", since a successful save turns every `R` into an `E`.

> **`P` does not answer the second question.** It sits in the same character position as F/E/R and replaces it, so a tuned parameter looks the same before and after a `/save` — which is a shame, because just after changing a value is exactly when you want to know whether it is safe yet. Until that is fixed, use `/save`'s own report, which counts the declarations it wrote.

The tag is a trailing **comment**, so a listing pastes straight back in as source: select it in one terminal, paste into another, and the tags are ignored along with everything else after `//`.

Settings

A **setting** is a value for something the program already declares, kept for *this unit* rather than for this build: a `#param` trimmed against this motor, a pin moved because this board is wired differently, a timer period found by experiment.

Set it at the prompt and save:

```
> Kp = 9
> Led.pin = 7
> T.period = 900
/save
```

That is all. It comes back after a power cycle, and — unlike a rule you typed — it also comes back after you **change the program and reflash**. A rule patch belongs to one firmware and is dropped when the image changes; a calibration belongs to the board and is not, so the two are kept in separate stores.

What may be a setting:

	recorded
a `#param` value	yes
`.pin .port .dir .pullup .pulldown .pwm .endian .period`	yes
`> Led = 1` — poking an output by hand	no
`.fired .rx .tx .dlc .len`	no

The line is configuration versus state. `> Led.pin = 7` says how the board is wired; `> Led = 1` turns the LED on to see which one it is. Only the first is worth carrying across a reboot, and keeping the second out is what makes the prompt safe to experiment at. A **rule** writing a config part is not recorded either — that is the program doing its job, not you configuring the unit.

An object's member is set by path, so a module costs nothing extra:

```
> sys.NodeID = 124
> sys.NodeName = "Node2"
```

Reading it back Three views answer different questions:

```
/list        what the SOURCE says      -- an overridden line is tagged S
/state       the LIVE value            -- what the unit is running
/settings    what is STORED            -- and whether each entry took
```

```
> /settings
Kp = 9
Led.pin = 7
sys.NodeID = 124
30 of 256 bytes, UNSAVED
```

`UNSAVED` means the store has changed since the last `/save` — the answer to "did my tuning actually take" that the `P` tag cannot give.

After a reflash An entry is applied only if the new firmware still has somewhere to put it, and `/settings` says so when it does not:

```
Kp = 9   // orphan                          the name is gone from this image
Kp = 9  // not applied: width or type moved  `#param Kp:16` became `#param Kp:32`
```

Neither is deleted. The next image may well reintroduce the name, and a calibration measured on real hardware is expensive to recreate — but an entry that is not in effect has to say so, or the store stops being trustworthy.

Setting a value **back to what the source says** removes its entry rather than storing a redundant override. Otherwise the day you change that default in the source it would be silently shadowed by an entry that matched it once.

> **A new `#param` is not a setting.** It is a declaration, so it goes in the rule patch and dies with it on reflash — correctly, since a parameter the running

firmware does not declare has nothing to configure. New parameters belong in the source. Parts cover what field work actually needs.

The store is a fixed area of RAM (256 bytes on a board, 1024 on the host) written to its own section of the EEPROM, ahead of the patch and with its own CRC — so it stays readable even when the patch does not. `doc/EEPROM.md` has the format.

Pause and Live

`/pause` stops the cycle entirely: no input, no rules, no output. The prompt keeps working, so you can inspect state and add declarations or rules; the rebuild is deferred until `/resume`.

`/live` freezes only the **rules**. Input is still sampled and output is still written, so the hardware stays connected while the program stands still — which is what you want when you are poking at pins by hand:

```
/live
> Led = 1          # actually lights the LED
> Btn              # reads the real pin
/resume
```

`/resume` leaves both modes.

Editing a Running Program

Declarations and rules can be added at any time — running, paused or live. A new rule is wired into the reactive graph at the next cycle boundary, so it starts firing without a restart. This is the intended way to patch a live system: since the last rule to write a field wins, adding a rule at the prompt overrides an earlier one.

If a line fails to parse, nothing is kept. A failure inside an unfinished `#module` rewinds the **whole module** and reports `Module aborted`, so the lines you type next are not silently swallowed by a module that can never be closed.

Disabling and Enabling Rules

`#disable` turns a single rule **off** by its number; `#enable` turns it back on. A disabled rule is skipped every cycle — nothing else changes.

`/list` numbers the rules in the trailing comment and marks a disabled one with `!`:

```
> /list
Led=1 ? Btn  // 1 R
Fan=1 ? Temp>30  // 2 R
> #disable 2
> /list
Led=1 ? Btn  // 1 R
Fan=1 ? Temp>30  // 2 R!      <- off
> #enable 2                   <- back on
```

You can name a list or a range, and sweep with a range:

```
#disable 3 5 7        # several at once
#disable 2-6          # an inclusive range
#enable  1-99      # re-enable everything (the top is clamped to the last rule)
```

What it is good for

- **Building past a bug.** A rule misbehaves — it fights another rule, trips on a bad sensor, floods an output. Instead of stopping the whole program to edit and reflash, switch that one rule off and keep running:

  ```
  > #disable 4              # silence the offending rule
  > Fan = Temp > 25         # add a corrected replacement rule
  ```

  The rest of the program is untouched. You have *built past* the bug live, and can take your time on a proper fix.

- **Patching firmware without reflashing.** Rule numbers run straight through the baked ROM rules and on into the ones you add at the prompt, so `#disable 2` can turn off a **firmware** rule just as easily as an interactive one. Silence a ROM rule, add a RAM rule in its place, and the board behaves the new way — no rebuild, no upload.

- **Bisecting behaviour.** Not sure which rule causes an effect? Disable a range, confirm it stops, then re-enable rules a few at a time to find the culprit.

- **Commissioning and testing.** Bring a machine up one rule at a time: disable everything, enable rules as each subsystem is verified.

Things worth knowing

- A disable **follows its rule**. Rule numbers shift when you insert or remove a rule, but a disable is remembered by identity across those edits — it stays on the rule you switched off, not on whatever number it happened to have.
- Disables **persist**. `/save` writes them to EEPROM and `/load` restores them, so a board comes back up with the same rules switched off. (If the program has changed so much that the numbers no longer line up, the saved set is dropped with a message rather than switching off the wrong rules.)
- `#disable 9` when there is no rule 9 is an **error** — naming a rule that does not exist is almost always a typo. A *range* that overshoots (`2-99`) is read as "to the end" and simply clamped.
- The first 128 rules each carry a switch; rules beyond that always run.

Direct Evaluation

In interactive mode you can:

```
> X + 1              # evaluate expression
> X = 5              # assign value directly
#variable Y = 10     # add new declaration
Y = X * 2        # add new rule
```

Output Latch

The latch controls whether outputs are written to hardware. Like an electronic latch:

- `/latch on` - Hold (latch) current output values. Outputs are calculated internally but not written to pins.
- `/latch off` - Release the latch. Outputs flow through to hardware normally.

Interactive mode starts with latch ON (outputs frozen). This is a safety feature - you can experiment without affecting hardware. Use `/latch off` when ready to activate outputs.

For programmatic control in running code, use the `latch()` function:

```
latch(1) ? Fault       // hold outputs on fault condition
latch(0) ? !Fault      // release when fault clears
```

Arduino Examples

Blink - Blinking LED

The classic Arduino example in CandySpeak:

```
#digital Led out 13
#timer Blink 500 = 1

Led = ~Led, Blink = 1 ? timeout(Blink)
```

LED toggles every 500 milliseconds. `Blink = 1` restarts the timer.

Button with LED

Light LED when button is pressed:

```
#digital Button in pullup 2
#digital Led out 13

Led = !Button
```

Button is active-low (pullup), so `!Button` is true when pressed.

Debouncer

Filter out contact bounce from a button:

```
#module Debouncer
#timer T 50
#variable Raw in integer
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
```

```
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
#digital Led out 13
#Debouncer db Raw <- !Button

Led = db.Out
```

The module waits 50ms after a change before accepting the new value.

Toggle - Toggle with Button

Press to toggle LED:

```
#module Debouncer
#timer T 50
#variable Raw in integer
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
#digital Led out 13
#variable LedState:1 = 0
#Debouncer db Raw <- !Button

LedState = ~LedState ? rising(db.Out)
Led = LedState
```

Dimmer - PWM Control

Adjust brightness with two buttons:

```
#digital BtnUp in pullup 2
#digital BtnDown in pullup 3
#analog Dimmer:8 out pwm 9
#variable Brightness:8 = 128
#timer Rep 100

Brightness = Brightness + 10 ? !BtnUp && Brightness < 245
Brightness = Brightness - 10 ? !BtnDown && Brightness > 10
Rep = 1 ? !BtnUp || !BtnDown
Dimmer = Brightness
```

Temperature Control

Simple thermostat with hysteresis:

```
#analog Sensor in A0
#digital Heater out 7
#variable Setpoint = 200
#variable Hysteresis = 10

Heater = 1 ? Sensor < Setpoint - Hysteresis
Heater = 0 ? Sensor > Setpoint + Hysteresis
```

The value 200 corresponds to approximately 20C with a typical NTC sensor.

Running Lights - Knight Rider

LEDs that move back and forth:

```
#digital Led0 out 2
#digital Led1 out 3
#digital Led2 out 4
#digital Led3 out 5
#digital Led4 out 6
#digital Led5 out 7
#digital Led6 out 8
#digital Led7 out 9

#timer Step 100 = 1
#variable Pos:4 = 0
#variable Dir:1 = 0

Pos = Pos + 1, Dir = 1 ? timeout(Step) && !Dir && Pos < 7
Pos = Pos - 1, Dir = 0 ? timeout(Step) && Dir && Pos > 0
Dir = 1 ? Pos == 7
Dir = 0 ? Pos == 0
Step = 1 ? timeout(Step)

Led0 = (Pos == 0)
Led1 = (Pos == 1)
Led2 = (Pos == 2)
Led3 = (Pos == 3)
Led4 = (Pos == 4)
Led5 = (Pos == 5)
Led6 = (Pos == 6)
Led7 = (Pos == 7)
```

Breathing LED

Smooth pulsing with PWM:

```
#analog Led:8 out pwm 9
#timer Step 20 = 1
#variable Brightness:8 = 0
#variable Dir:1 = 0

Brightness = Brightness + 5 ? timeout(Step) && !Dir
Brightness = Brightness - 5 ? timeout(Step) && Dir
Dir = 1 ? Brightness >= 250
Dir = 0 ? Brightness <= 5
Step = 1 ? timeout(Step)
Led = Brightness
```

Traffic Light

Sequential traffic light:

```
#digital Red out 2
#digital Yellow out 3
#digital Green out 4

#timer Phase 1000 = 1
#variable State:2 = 0

State = 1 ? timeout(Phase) && State == 0
State = 2 ? timeout(Phase) && State == 1
State = 3 ? timeout(Phase) && State == 2
State = 0 ? timeout(Phase) && State == 3
Phase = 1 ? timeout(Phase)

Red = (State == 0 || State == 1)
Yellow = (State == 1 || State == 3)
Green = (State == 2)
```

Sequence: Red -> Red+Yellow -> Green -> Yellow -> Red...

The same machine reads far more clearly with named `#states` — each phase is a state, and a transition is one assignment to `State`:

```
#digital Red    out 2
#digital Yellow out 3
#digital Green  out 4
#timer   Phase  1000 = 1
#states  red redyellow green yellow

#in INIT
    State = red
#end
#in red
    Red=1, Yellow=0, Green=0
    State = redyellow ? timeout(Phase)
```

```
#end
#in redyellow
    Red=1, Yellow=1, Green=0
    State = green ? timeout(Phase)
#end
#in green
    Red=0, Yellow=0, Green=1
    State = yellow ? timeout(Phase)
#end
#in yellow
    Red=0, Yellow=1, Green=0
    State = red ? timeout(Phase)
#end

#in red redyellow green yellow
    Phase = 1 ? timeout(Phase)    // restart the timer in every driving phase
#end
```

Each state drives the three lamps and, on `timeout(Phase)`, hands over to the next state. The output pattern for a phase settles the cycle after the state is entered — invisible for a one-second light, and the structure scales to far more states without the tangle of `State == n` conditions.

Baking a Program into Firmware

A CandySpeak program can be *baked into the firmware* instead of being parsed at start-up. `-C` writes the parsed program as C source — `rom.c`, which defines the `rom_decl[]`, `rom_instr[]` and `rom_str[]` arrays:

```
./csp -C -n program.csp > rom.c
```

This `rom.c` is **compiled and linked** into the firmware (not `#included` as a header). At start-up `csp_load_rom()` points the runtime at it, and the program runs in place from flash.

What this buys you — and what it doesn't. The baked program runs on the *same* virtual machine over the *same* bytecode as a program typed at runtime; it is **not** faster (uncached flash reads can even make it a touch slower than a RAM-resident program). The real gains are elsewhere:

- **Saves RAM** — the program lives in flash, not in the scarce RAM.
- **No parsing at boot** — the program is ready immediately; the parser need not even be linked in.
- **It is firmware** — it survives without an EEPROM save.

Writing a Sketch

There is no auto-generated `.ino` wrapper. Today there are two practical paths, both starting from a fork/checkout of the CandySpeak sources:

1. **Hack the board glue.** Edit `csp_arduino.c` (the platform layer: `csp_board_*`, `setup()`/`loop()`). This is the easiest route when you are adding or wiring up hardware. See `CandySpeak/CandySpeak.ino` for a complete, working example (Circuit Playground Express) — it shows the real cycle: `csp_input()` → `csp_cycle()` → `csp_commit()` → `csp_output()`, wrapped in `csp_rt_init` / `csp_load_rom` / `csp_rt_start` / `csp_setup(&state)` during `setup()`.
2. **Freeze a program.** Generate `rom.c` with `-C` as above, drop it into the build, and flash. The sketch loads it via `csp_load_rom()`.

The firmware build needs the core sources — `csp.h`, `csp_config.h`, `csp_rt.c`, `csp_print.c`, `csp_strings.c`, `csp_parse.c`, `csp_eeprom.c`, `bitpack.h`, `csp_fixpoint.h` — plus your board glue and, for the frozen-program route, `rom.c`.

> This is the current, hands-on workflow; a smoother sketch story is still to be designed.

PDF Generation

This manual can be converted to PDF with pandoc:

```
# Install pandoc and LaTeX
sudo apt install pandoc texlive-xetex fonts-dejavu

# Generate PDF
pandoc doc/manual_en.md -o doc/manual_en.pdf \
  --pdf-engine=xelatex \
  --toc \
  --toc-depth=2 \
  -V colorlinks=true \
  -V linkcolor=blue
```

Appendix: Quick Reference

Declarations

```
#variable <name>[:<bits>] [type] [= value]          // bits 1..32, typeless = signed
#variable  <name>:<bits>  [big|little]  bind  <buffer>[<a>..<b>]     //  bit-
field view
#local  <name>[:<bits>]  [type]  =  <expr>      //  a  named  FORMULA,  same-
cycle, no assign
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
#analog <name>[:<resolution>] [in|out] [pwm] [integer|unsigned] [<port>:]<pin>
#timer <name> <period_ms|param> [= 1]
#constant <name> = <value>
#param <name>[:<bits>] [type] = <value>  // a constant that does NOT fold:
                                        // set it from the prompt, re-declare
                                       // it to load a setting, /save keeps it
#buffer <name>:<bytes> [type]           // shared storage (size in BYTES)
```

```
#buffer <name>:<bytes> [in|out] can <id>  // CAN frame (size in BYTES)
#field <name>:<bits> [type] [big|little] <frame>[<a>..<b>]  // field of a frame
#states <name> ...                       // INIT/NORMAL/FAILSAFE implicit
#module <name> ... #end
```

Arrays

```
#variable Acc[3] = 0                     // N declarations, one per element
#constant CT[10] = { -100, -81, 31 }     // init list, one value per element
#digital  D[5]  in 0:1..3,7,9            // pin list: 1,2,3,7,9
#analog   P[10]:16 out unsigned 9:0..9   // pin range: one pin per element
#digital  E[4]  in 0:2,1:5,2:6,3:7       // a port names the pins after it

x = A[I]                      // runtime index: checked every cycle
A[(I + 1) % 10] = v           // ...on the left as well
y = CT[3]                    // constant index: free, checked at compile time
z = A                         // no subscript = element 0
```

Buffers

```
#buffer Buf:3             // 3 BYTES of shared storage (size is always bytes)
Buf[0] = 52                 // byte access (index = byte)
X = Buf[0..1]               // byte range -> one value (little-endian)

#variable F:8     bind Buf[0..7]    // bind range = bits
#variable G:10 big bind Buf[8..17]  // big-endian bit-field
```

CAN

```
#buffer F201:8 in  can 0x201     // 8 BYTES (the DLC), received
#buffer Out:8  out can 0x200     // transmitted
#field Speed:16 unsigned F201[0..15]   // field: a bit view into the frame

Speed = v                      // writing a field sends the frame (event PDO)
Out.tx  = 1 ? timeout(Beat)      // cyclic PDO: send even if unchanged
Out.dlc = 3                      // send 3 bytes instead of 8

println(Speed) ? F201.rx        // .rx: true the cycle the frame is readable
Got = F201.dlc                   // bytes the sender actually sent
Id  = F201.id                    // frame id

./csp --can=vcan0 prog.csp       // Linux: SocketCAN
```

Module Instantiation

```
#<Module> <instance> [<init>]*

Init forms (can be mixed):
```

```
  field = value      // static init of the field's VALUE, runs once in INIT
  field.part = value // set a field attribute once
  field <- expr      // reactive connection (seeded once at start-up)

 D.pin = 2        // place a #digital/#analog  (NOT `D = 2`, that is a value)
  D.port = 1
  T.period = 500
```

Rules

```
<actions> ? <condition>
action1, action2 ? condition
variable = expression ? condition   // regular assignment
variable <- expression              // reactive (runs on change)
variable <- expression ? condition  // reactive with condition
Timer = 1 ? condition               // start timer
```

Timer Functions

```
timeout(T)    // true one cycle at timeout
elapsed(T)    // ms since start
progress(T)   // 0-100% of period
T = 1         // start/restart timer
```

Edge Detection

```
changed(x)    // value changed
rising(x)     // 0 -> 1
falling(x)    // 1 -> 0
```

States

```
#states A B C                // INIT, NORMAL and FAILSAFE always exist

#in A                        // rules active only while State == A
    ...
    State = B ? cond         // transition
#end

#in FAILSAFE                 // the safe island: entered once, never left
    ...                      // (only a reset releases it)
#end
```

Parts

```
<resource>.<part>            // read or write an attribute
D.pin = 17                   // .val .pin .port .dir .pwm
T.period = 500               // .pullup .pulldown .period .fired
```

```
Ready = 1 ? T.fired
F.id  F.rx  F.tx  F.dlc      // CAN frame parts (also via any of its fields)
```

Interactive

```
/pause  /resume  /live        // freeze all / continue / freeze rules only
/latch on|off                 // hold or release outputs
/list  /state  /memory        // inspect
/save  /load                  // EEPROM
#disable 2   #enable 2        // switch a rule off / on by number
#disable 2-6                  // a list or inclusive range
```

Packing and Unpacking

```
Buf <<= A B C             // pack fields into a buffer (ascending bits)
Buf <<= (X+4):3 X:2 6:3   // field := <expr>[:<bits>]
Buf >>= RA:3 RB:2 RC:3    // unpack: each var takes the next field
```